Listando usuários

```python
class ListarUsuario(generics.ListCreateAPIView):
    # Define a consulta para recuperar todos os usuários do banco de dados
    queryset = Usuario.objects.all()

    # Define o serializador a ser utilizado para serializar e desserializar os dados do usuário
    serializer_class = UsuarioSerializer

    # Método GET para exibir a lista de usuários
    def get(self, request, *args, **kwargs):
        # Obtém a lista de usuários do banco de dados
        usuarios = self.get_queryset()
        # Renderiza o template 'listarusuario.html', passando a lista de usuários como contexto
        return render(request, 'listarusuario.html', {'usuarios': usuarios})

    # Método POST para processar o formulário de criação de usuários
    def post(self, request, *args, **kwargs):
        # Inicializa o serializador com os dados da requisição
        serializer = self.serializer_class(data=request.data)

        # Verifica se os dados do formulário são válidos
        if serializer.is_valid():
            # Redireciona para a página de listagem de usuários após a criação bem-sucedida
            return redirect('listar')
```

Vamos começar direto da view dessa vez resgatamos todos os usuários do nosso banco, e listamos no html ‘listarusuario.html”

URLS.PY

```python
urlpatterns = [
    path('',views.CadastrarUsuario.as_view(),name='cadastro'),
    path('listar_usuarios',views.ListarUsuario.as_view(),name='listar'),

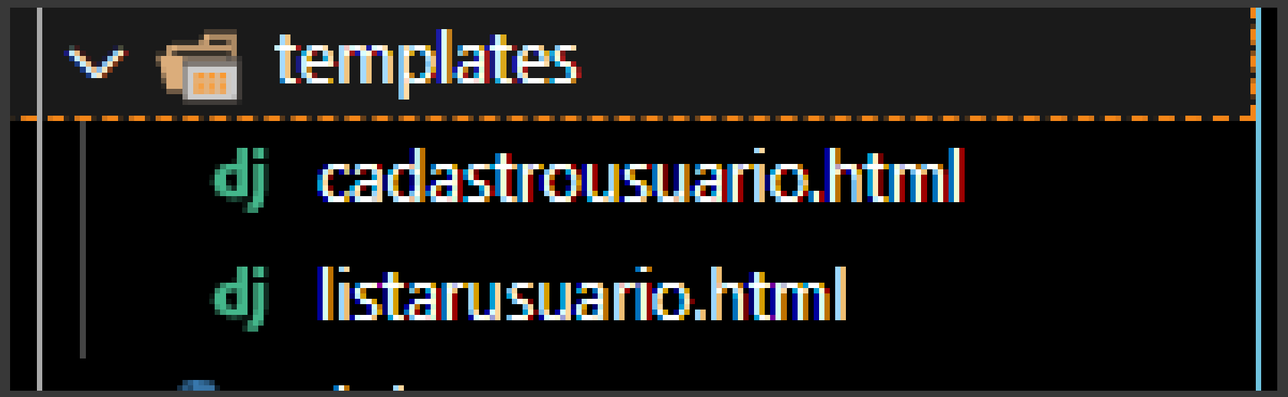


]

if settings.DEBUG:
```

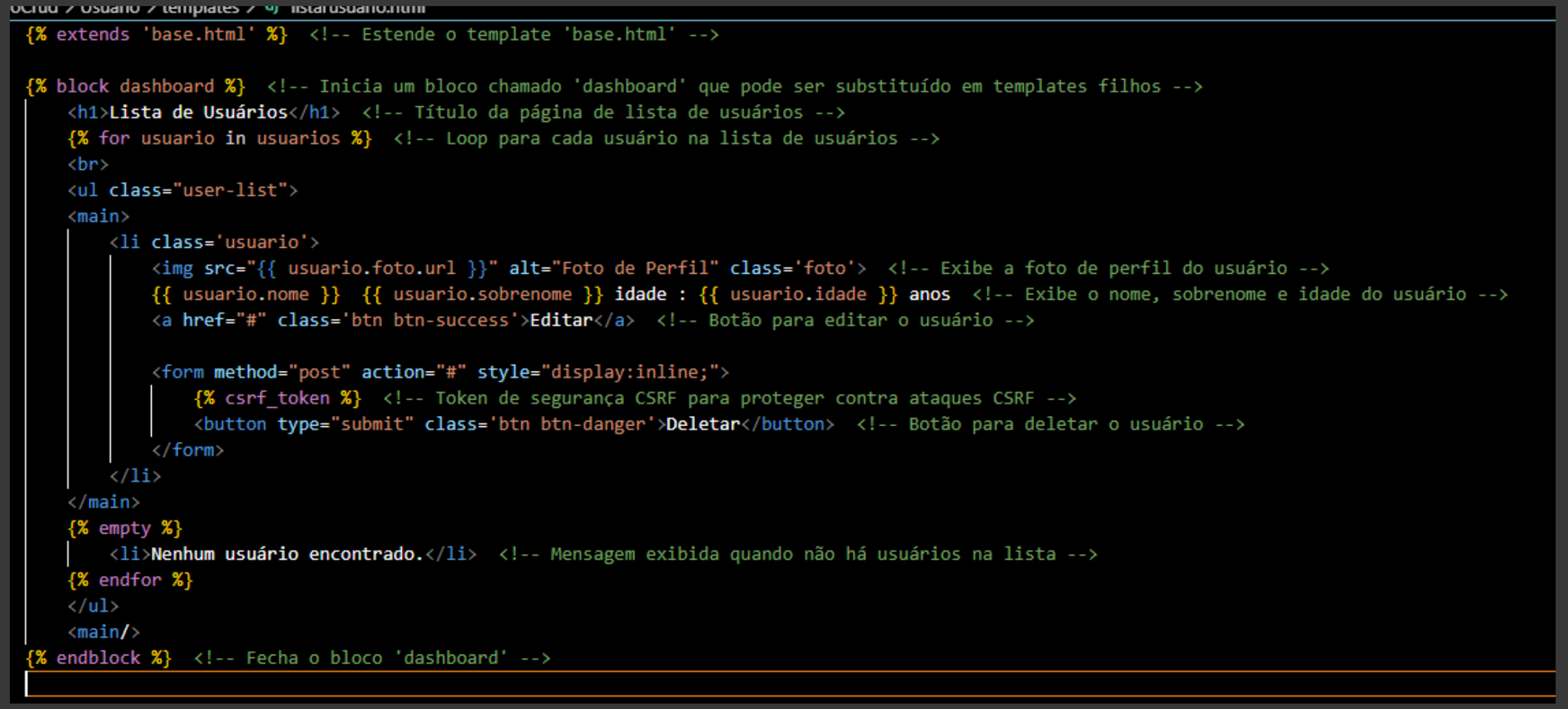
A URL funciona de maneira semelhante, mas dessa vez, não usamos uma String vazia, mas sim uma URL correspondente a função, a String vazia representava a pagina inicial, já essa, uma subpagina

LISTAR.HTML



Depois de ter feito a VIEW e a URL, podemos já passar a criar um HTML, que vai na mesma pasta do de cadastro.

Por dentro do HTML



```
{% extends 'base.html' %}  <!-- Estende o template 'base.html' -->

{% block dashboard %}  <!-- Inicia um bloco chamado 'dashboard' que pode ser substituído em templates filhos -->
    <h1>Lista de Usuários</h1>  <!-- Título da página de lista de usuários -->
    {% for usuario in usuarios %}  <!-- Loop para cada usuário na lista de usuários -->
    <br>
    <ul class="user-list">
    <main>
        <li class='usuario'>
            <img src="{{ usuario.foto.url }}" alt="Foto de Perfil" class='foto'>  <!-- Exibe a foto de perfil do usuário -->
            {{ usuario.nome }}  {{ usuario.sobrenome }} idade : {{ usuario.idade }} anos  <!-- Exibe o nome, sobrenome e idade do usuário -->
            <a href="#" class='btn btn-success'>Editar</a>  <!-- Botão para editar o usuário -->

            <form method="post" action="#" style="display:inline;">
                {% csrf_token %}  <!-- Token de segurança CSRF para proteger contra ataques CSRF -->
                <button type="submit" class='btn btn-danger'>Deletar</button>  <!-- Botão para deletar o usuário -->
            </form>
        </li>
    </main>
    {% empty %}
        <li>Nenhum usuário encontrado.</li>  <!-- Mensagem exibida quando não há usuários na lista -->
    {% endfor %}
    </ul>
    <main/>
{% endblock %}  <!-- Fecha o bloco 'dashboard' -->
```

Nesse HTML podemos ver coisas interessantes, como o uso de uma estrutura de repetição for para exibir os usuários acessamos os dados dos usuários a instancia usuário.

e assim deve estar a tela de listar usuários

